

EE652
IC Design Laboratory

RISCV Code Simulation Results

Group 3

Shah Tirth
Rishank Soni
Anirudh Mittal
Shail Bhargavkumar Joshi
Arjun Anand Mallya
Krish Patel
Kshitij Agarwal
Shriniket Behra
Moksh Jindal
Mayank Kumbhat

Program 1: Max Number

C Code:

```
int max(int a, int b) {  
    return (a > b) ? a : b;  
}
```

```
int main() {  
    int m;  
    int a = 1;  
    int b = 49;  
    int c = 4;  
    int d = 3;  
    int e = 67;  
    m = max(a,b);  
    m = max(m,c);  
    m = max(m,d);  
    m = max(m,e);  
  
    return m;  
}
```

Assembly code:

program.elf: file format elf32-littleriscv

Disassembly of section .text:

```
00000000 <_start>:  
0: 03c000ef        jal    3c <main>  
4: 0000006f        j      4 <_start+0x4>  
  
00000008 <max>:  
8: fe010113        addi   sp,sp,-32
```

c: 00812e23	sw	s0,28(sp)
10: 02010413	addi	s0,sp,32
14: fea42623	sw	a0,-20(s0)
18: feb42423	sw	a1,-24(s0)
1c: fec42703	lw	a4,-20(s0)
20: fe842783	lw	a5,-24(s0)
24: 00e7d463	bge	a5,a4,2c <max+0x24>
28: 00070793	mv	a5,a4
2c: 00078513	mv	a0,a5
30: 01c12403	lw	s0,28(sp)
34: 02010113	addi	sp,sp,32
38: 00008067	ret	

0000003c <main>:

3c: fd010113	addi	sp,sp,-48
40: 02112623	sw	ra,44(sp)
44: 02812423	sw	s0,40(sp)
48: 03010413	addi	s0,sp,48
4c: 00100793	li	a5,1
50: fef42623	sw	a5,-20(s0)
54: 03100793	li	a5,49
58: fef42423	sw	a5,-24(s0)
5c: 00400793	li	a5,4
60: fef42223	sw	a5,-28(s0)
64: 00300793	li	a5,3
68: fef42023	sw	a5,-32(s0)
6c: 04300793	li	a5,67
70: fcf42e23	sw	a5,-36(s0)
74: fe842583	lw	a1,-24(s0)
78: fec42503	lw	a0,-20(s0)
7c: f8dff0ef	jal	8 <max>
80: fca42c23	sw	a0,-40(s0)
84: fe442583	lw	a1,-28(s0)
88: fd842503	lw	a0,-40(s0)
8c: f7dff0ef	jal	8 <max>
90: fca42c23	sw	a0,-40(s0)
94: fe042583	lw	a1,-32(s0)
98: fd842503	lw	a0,-40(s0)
9c: f6dff0ef	jal	8 <max>
a0: fca42c23	sw	a0,-40(s0)

a4: fdc42583	lw	a1,-36(s0)
a8: fd842503	lw	a0,-40(s0)
ac: f5dff0ef	jal	8 <max>
b0: fca42c23	sw	a0,-40(s0)
b4: fd842783	lw	a5,-40(s0)
b8: 00078513	mv	a0,a5
bc: 02c12083	lw	ra,44(sp)
c0: 02812403	lw	s0,40(sp)
c4: 03010113	addi	sp,sp,48
c8: 00008067	ret	

Hex Instructions:

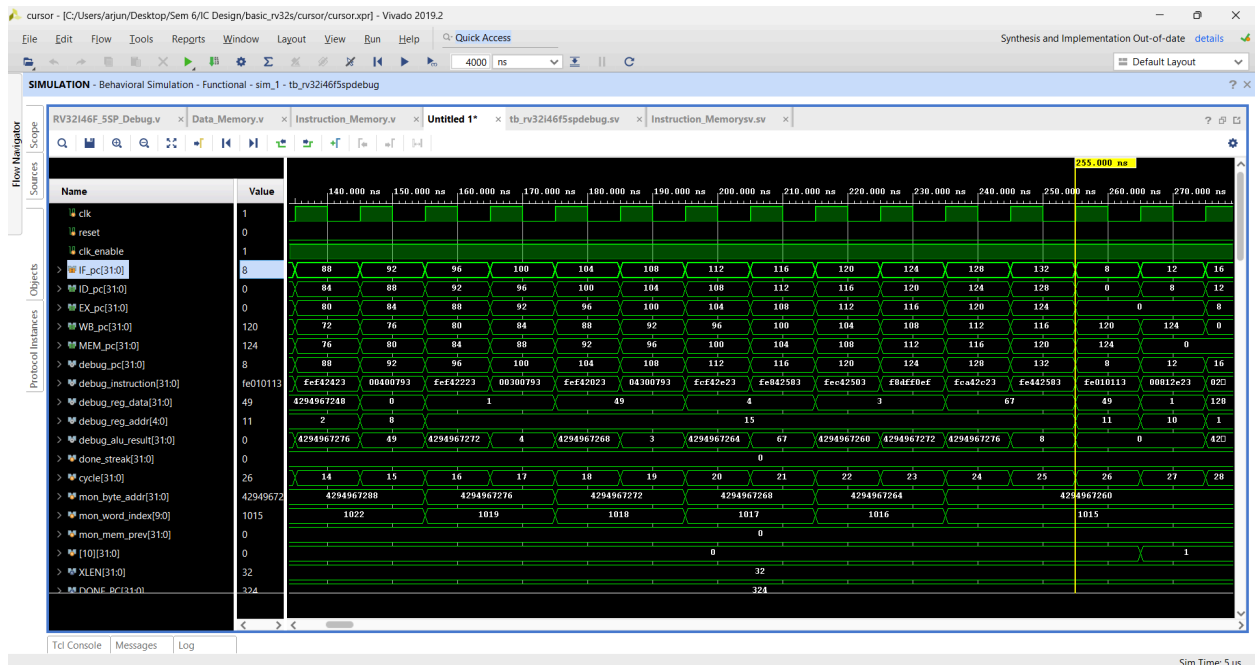
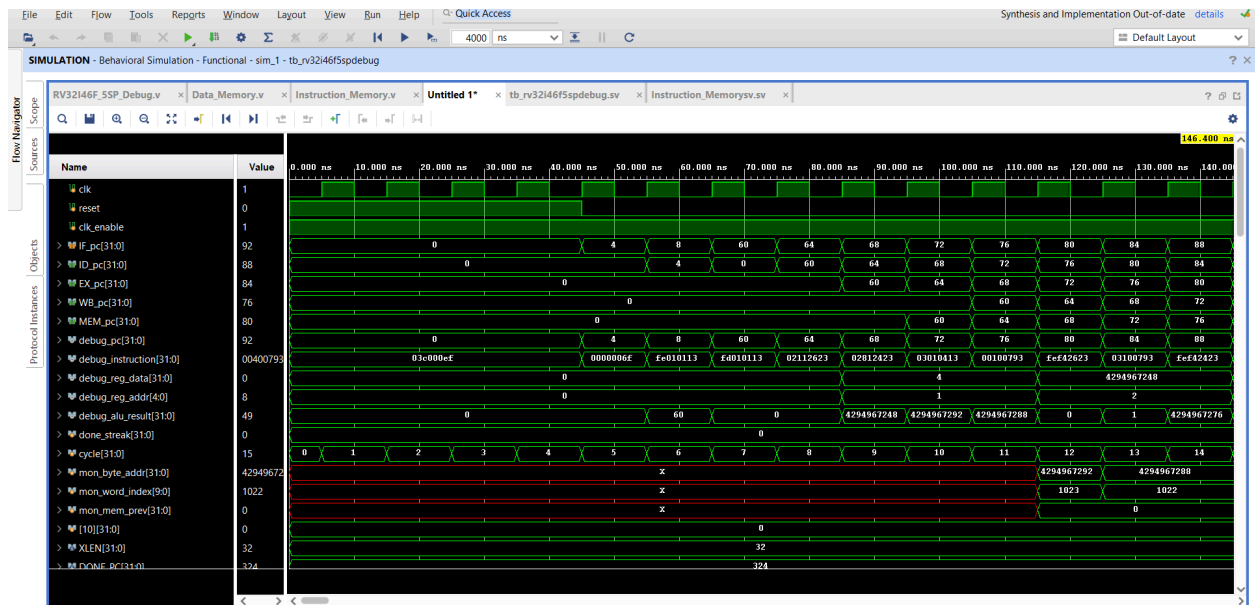
```

data[0] = 32'h03c000ef;
data[1] = 32'h0000006f;
data[2] = 32'hfe010113;
data[3] = 32'h00812e23;
data[4] = 32'h02010413;
data[5] = 32'hfea42623;
data[6] = 32'hfeb42423;
data[7] = 32'hfec42703;
data[8] = 32'hfe842783;
data[9] = 32'h00e7d463;
data[10] = 32'h00070793;
data[11] = 32'h00078513;
data[12] = 32'h01c12403;
data[13] = 32'h02010113;
data[14] = 32'h00008067;
data[15] = 32'hfd010113;
data[16] = 32'h02112623;
data[17] = 32'h02812423;
data[18] = 32'h03010413;
data[19] = 32'h00100793;
data[20] = 32'hfef42623;
data[21] = 32'h03100793;
data[22] = 32'hfef42423;
data[23] = 32'h00400793;
data[24] = 32'hfef42223;
data[25] = 32'h00300793;
data[26] = 32'hfef42023;

```

```
data[27] = 32'h04300793;  
data[28] = 32'hfcf42e23;  
data[29] = 32'hfe842583;  
data[30] = 32'hfec42503;  
data[31] = 32'hf8dff0ef;  
data[32] = 32'hfca42c23;  
data[33] = 32'hfe442583;  
data[34] = 32'hfd842503;  
data[35] = 32'hf7dff0ef;  
data[36] = 32'hfca42c23;  
data[37] = 32'hfe042583;  
data[38] = 32'hfd842503;  
data[39] = 32'hf6dff0ef;  
data[40] = 32'hfca42c23;  
data[41] = 32'hfdc42583;  
data[42] = 32'hfd842503;  
data[43] = 32'hf5dff0ef;  
data[44] = 32'hfca42c23;  
data[45] = 32'hfd842783;  
data[46] = 32'h00078513;  
data[47] = 32'h02c12083;  
data[48] = 32'h02812403;  
data[49] = 32'h03010113;  
data[50] = 32'h00008067;
```

Simulation Waveform:



Tcl Console Messages Log

Sim Time: 5 ns



Analysis:

The code takes 109 cycles to execute. It computes the max of 5 numbers. Based on the assembly, we observe that the max function has 15 instructions which will take 15 cycles to execute. Also we are calculating the max 4 times, contributing 60 cycles. In total, other than this there are 35 instructions. Thus, In an ideal case it will take 95 cycles. However, every jump and link and return instruction stalls the pipeline for 1 cycle to flush the pipeline and fetch the correct instructions. From the code we see that this will lead to 10 stalls. Also, there is 1 load-use stall per execution of the max function. Since it is called 4 times, it leads to 4 stalls. Thus, the total cycles is $60 + 35 + 10 + 4 = 109$ cycles. As seen in the simulation waveforms, the return register (x10) gets the value of 67 at the end of execution confirming the correctness of the code.

Program 2: Sorting

C Code:

```
int main() {
    int arr[5];

    // initialize values
    arr[0] = 1;
    arr[1] = 49;
    arr[2] = 4;
    arr[3] = 3;
    arr[4] = 67;

    // bubble sort
    for (int i = 0; i < 5 - 1; i++) {
        for (int j = 0; j < 5 - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }

    return arr[4]; // largest element after sorting
}
```

Assembly code:

program.elf: file format elf32-littleriscv

Disassembly of section .text:

00000000 <_start>:

0:	008000ef	jal	8 <main>
4:	0000006f	j	4 <_start+0x4>

00000008 <main>:

8:	fd010113	addi	sp,sp,-48
c:	02812623	sw	s0,44(sp)
10:	03010413	addi	s0,sp,48
14:	00100793	li	a5,1
18:	fcf42823	sw	a5,-48(s0)
1c:	03100793	li	a5,49
20:	fcf42a23	sw	a5,-44(s0)
24:	00400793	li	a5,4
28:	fcf42c23	sw	a5,-40(s0)
2c:	00300793	li	a5,3
30:	fcf42e23	sw	a5,-36(s0)
34:	04300793	li	a5,67
38:	fef42023	sw	a5,-32(s0)
3c:	fe042623	sw	zero,-20(s0)
40:	0c80006f	j	108 <main+0x100>
44:	fe042423	sw	zero,-24(s0)
48:	0a00006f	j	e8 <main+0xe0>
4c:	fe842783	lw	a5,-24(s0)
50:	00279793	slli	a5,a5,0x2
54:	ff078793	addi	a5,a5,-16
58:	008787b3	add	a5,a5,s0
5c:	fe07a703	lw	a4,-32(a5)
60:	fe842783	lw	a5,-24(s0)
64:	00178793	addi	a5,a5,1
68:	00279793	slli	a5,a5,0x2
6c:	ff078793	addi	a5,a5,-16
70:	008787b3	add	a5,a5,s0
74:	fe07a783	lw	a5,-32(a5)
78:	06e7d263	bge	a5,a4,dc <main+0xd4>
7c:	fe842783	lw	a5,-24(s0)
80:	00279793	slli	a5,a5,0x2
84:	ff078793	addi	a5,a5,-16
88:	008787b3	add	a5,a5,s0
8c:	fe07a783	lw	a5,-32(a5)

90:	fef42223	sw	a5,-28(s0)
94:	fe842783	lw	a5,-24(s0)
98:	00178793	addi	a5,a5,1
9c:	00279793	slli	a5,a5,0x2
a0:	ff078793	addi	a5,a5,-16
a4:	008787b3	add	a5,a5,s0
a8:	fe07a703	lw	a4,-32(a5)
ac:	fe842783	lw	a5,-24(s0)
b0:	00279793	slli	a5,a5,0x2
b4:	ff078793	addi	a5,a5,-16
b8:	008787b3	add	a5,a5,s0
bc:	fee7a023	sw	a4,-32(a5)
c0:	fe842783	lw	a5,-24(s0)
c4:	00178793	addi	a5,a5,1
c8:	00279793	slli	a5,a5,0x2
cc:	ff078793	addi	a5,a5,-16
d0:	008787b3	add	a5,a5,s0
d4:	fe442703	lw	a4,-28(s0)
d8:	fee7a023	sw	a4,-32(a5)
dc:	fe842783	lw	a5,-24(s0)
e0:	00178793	addi	a5,a5,1
e4:	fef42423	sw	a5,-24(s0)
e8:	00400713	li	a4,4
ec:	fec42783	lw	a5,-20(s0)
f0:	40f707b3	sub	a5,a4,a5
f4:	fe842703	lw	a4,-24(s0)
f8:	f4f74ae3	blt	a4,a5,4c <main+0x44>
fc:	fec42783	lw	a5,-20(s0)
100:	00178793	addi	a5,a5,1
104:	fef42623	sw	a5,-20(s0)
108:	fec42703	lw	a4,-20(s0)
10c:	00300793	li	a5,3
110:	f2e7dae3	bge	a5,a4,44 <main+0x3c>
114:	fe042783	lw	a5,-32(s0)
118:	00078513	mv	a0,a5
11c:	02c12403	lw	s0,44(sp)
120:	03010113	addi	sp,sp,48
124:	00008067	ret	

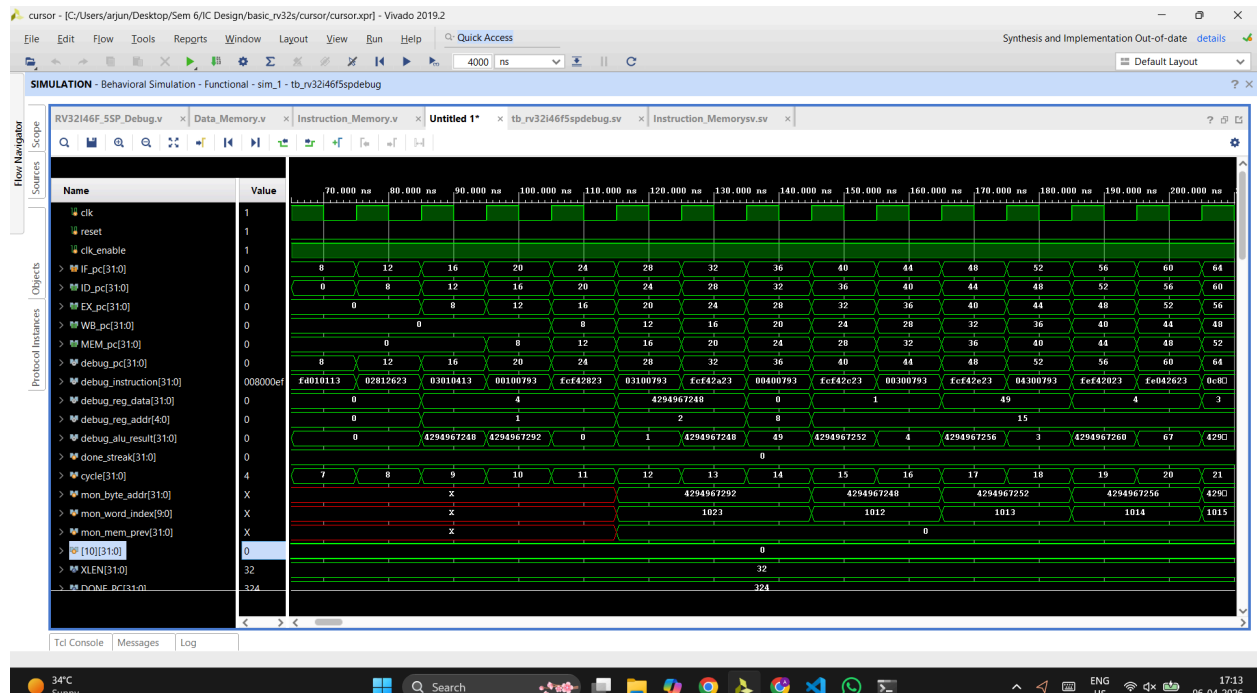
Hex Instructions:

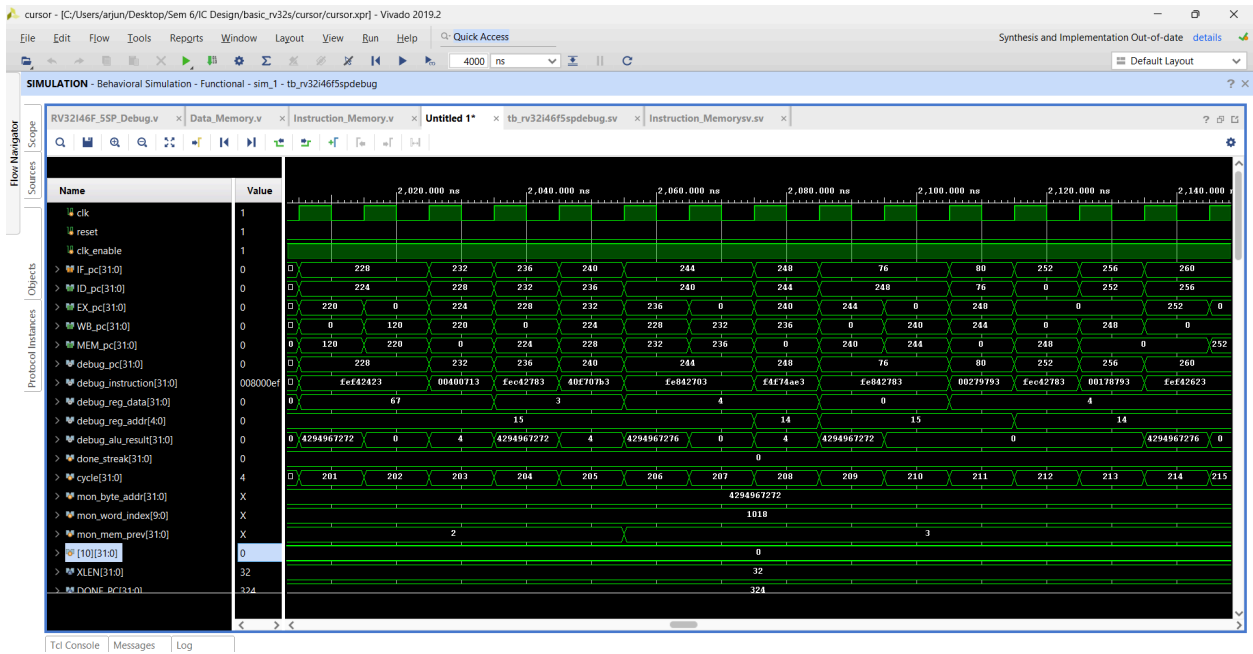
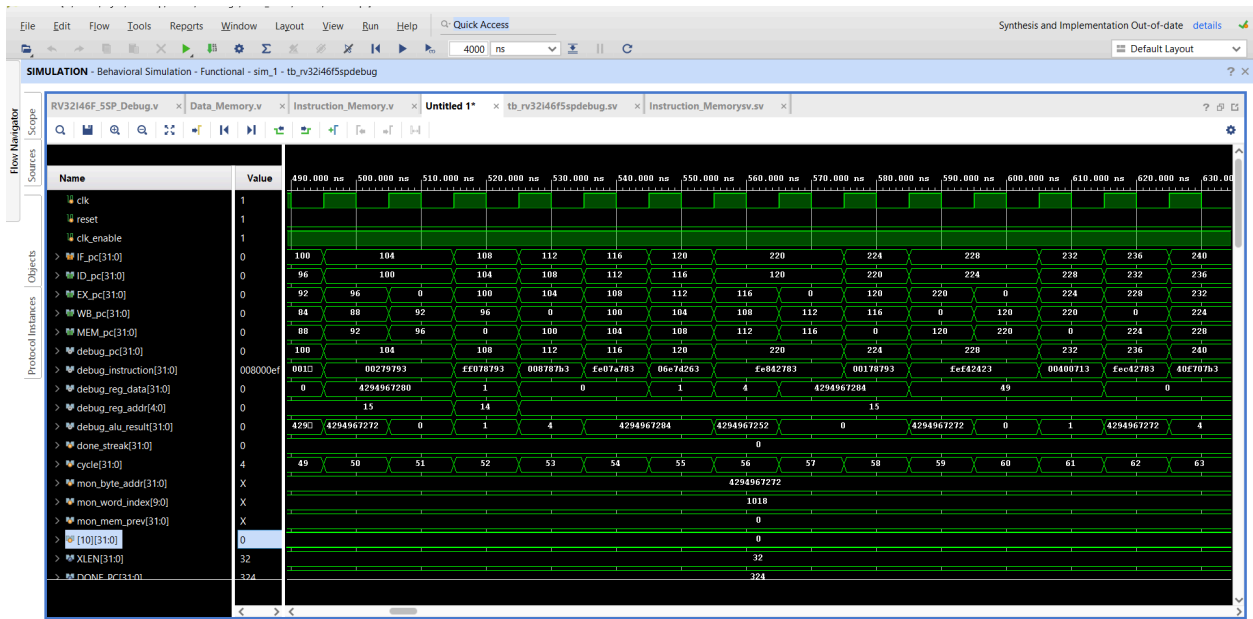
data[0] = 32'h008000ef;

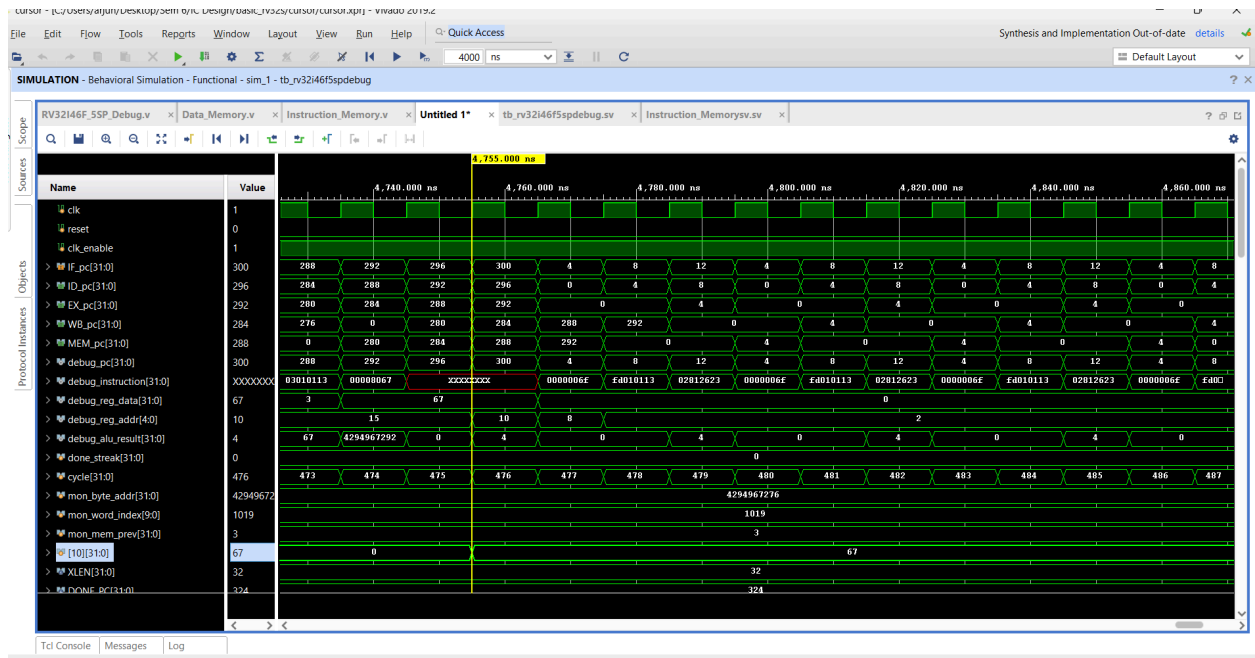
```
data[1] = 32'h0000006f;  
data[2] = 32'hfd010113;  
data[3] = 32'h02812623;  
data[4] = 32'h03010413;  
data[5] = 32'h00100793;  
data[6] = 32'hfcf42823;  
data[7] = 32'h03100793;  
data[8] = 32'hfcf42a23;  
data[9] = 32'h00400793;  
data[10] = 32'hfcf42c23;  
data[11] = 32'h00300793;  
data[12] = 32'hfcf42e23;  
data[13] = 32'h04300793;  
data[14] = 32'hfef42023;  
data[15] = 32'hfe042623;  
data[16] = 32'h0c80006f;  
data[17] = 32'hfe042423;  
data[18] = 32'h0a00006f;  
data[19] = 32'hfe842783;  
data[20] = 32'h00279793;  
data[21] = 32'hff078793;  
data[22] = 32'h008787b3;  
data[23] = 32'hfe07a703;  
data[24] = 32'hfe842783;  
data[25] = 32'h00178793;  
data[26] = 32'h00279793;  
data[27] = 32'hff078793;  
data[28] = 32'h008787b3;  
data[29] = 32'hfe07a783;  
data[30] = 32'h06e7d263;  
data[31] = 32'hfe842783;  
data[32] = 32'h00279793;  
data[33] = 32'hff078793;  
data[34] = 32'h008787b3;  
data[35] = 32'hfe07a783;  
data[36] = 32'hfef42223;  
data[37] = 32'hfe842783;  
data[38] = 32'h00178793;  
data[39] = 32'h00279793;  
data[40] = 32'hff078793;
```

```
data[41] = 32'h008787b3;  
data[42] = 32'hfe07a703;  
data[43] = 32'hfe842783;  
data[44] = 32'h00279793;  
data[45] = 32'hff078793;  
data[46] = 32'h008787b3;  
data[47] = 32'hfee7a023;  
data[48] = 32'hfe842783;  
data[49] = 32'h00178793;  
data[50] = 32'h00279793;  
data[51] = 32'hff078793;  
data[52] = 32'h008787b3;  
data[53] = 32'hfe442703;  
data[54] = 32'hfee7a023;  
data[55] = 32'hfe842783;  
data[56] = 32'h00178793;  
data[57] = 32'hfef42423;  
data[58] = 32'h00400713;  
data[59] = 32'hfec42783;  
data[60] = 32'h40f707b3;  
data[61] = 32'hfe842703;  
data[62] = 32'hf4f74ae3;  
data[63] = 32'hfec42783;  
data[64] = 32'h00178793;  
data[65] = 32'hfef42623;  
data[66] = 32'hfec42703;  
data[67] = 32'h00300793;  
data[68] = 32'hf2e7dae3;  
data[69] = 32'hfe042783;  
data[70] = 32'h00078513;  
data[71] = 32'h02c12403;  
data[72] = 32'h03010113;  
data[73] = 32'h00008067;
```

Simulation Waveform:







Analysis:

The code takes 473 cycles to execute. It computes the sorted array of 5 numbers using bubble sort and returns the maximum value. Based on the assembly, we observe that the inner loop (comparing and swapping array elements) has roughly 30 instructions which will take 30 cycles to execute. Also we are executing this inner loop 10 times (4 + 3 + 2 + 1 iterations), contributing 300 cycles. In total, other than this there are roughly 80 instructions used for array initialization, outer loop overhead, and function setup/return. Thus, In an ideal case it will take 380 cycles. However, every jump, taken branch, and return instruction stalls the pipeline for 1 cycle to flush the pipeline and fetch the correct instructions. From the code we see that the repeated looping and branching will lead to about 46 stalls. Also, there are multiple load-use stalls per execution of the inner loop due to loading array elements from memory and immediately comparing them. Since it accesses memory constantly over the 10 iterations, it leads to roughly 47 stalls. Thus, the total cycles is $300 + 80 + 46 + 47 = 473$ cycles. As seen in the simulation waveforms, the return register (x10) gets the value of 67 (0x43 in hex) at the end of execution confirming the correctness of the code.